Project 2: Classifying Application Risk

Packaged Model Instructions

Requirements: Python 3, Pandas, Matplotlib, Scikit-learn and XGBoost. **Firstly**, make sure project2_model.py, test_packaged_model.py, project2_raw_data.zip, and feature_name_to_number_mapping.csv are in the same project folder. **Secondly**, run project2_model.py, this processes the raw data, creates the modelling dataset, trains and evaluates the models, selects the final validation threshold and saves the packaged model as data/processed/packaged_model.pkl. **Thirdly**, run test_packaged_model.py, this loads packaged_model.pkl without retraining and tests it on a selected sample, the sample can be changed using selected_txt_file, start_row, and end_row. **Fourthly** the input rows must have the same format as the raw data (sparse feature_number:feature_value), excluding the risk score. Missing raw features can be left missing as XGBoost handles missing values natively. **Input checks:** Empty rows, rows with no valid mapped features, incorrectly formatted tokens, and unknown feature numbers are handled through errors or warnings rather than being silently converted to all-zero inputs. **Outputs:** The test script outputs the source file, row number, predicted high-risk probability, predicted class, and readable risk classification. Sample predictions are saved as data/processed/sample_classifications.csv.

Data Preparation Summary

The raw data is stored in project2_raw_data.zip as monthly text files, where each row represents one executable application. The first value in each row is the risk score, followed by sparse feature_number:feature_value pairs. The target variable, risk_level, was created from the risk score: scores of 0.30 or above were labelled high risk, while scores below 0.30 were labelled low risk, as defined in the specification. The raw sparse features were converted into a flat modelling dataset using feature_name_to_number_mapping.csv, which maps each feature number to its feature name, type, and subtype. The feature names were cleaned and were named with their feature number followed by their feature name, this allows the duplicate features to be separate, so they retain their individual values. Instead of setting missing raw feature values to zero, they were left as NaN as XGBoost can handle missing values natively. The code checks mapper values, duplicate feature numbers, duplicate feature names, column completeness, and numeric data types. Additional features were created, including total feature activity, active feature count, feature-type totals, feature-subtype totals, dominant feature-type share, and dominant feature-subtype share. The same feature-engineering function is used for both training and packaged inference. The final modelling dataset is saved as modelling_data.csv, and the cleaned feature dictionary is saved as feature_dictionary.csv.

Model Summary

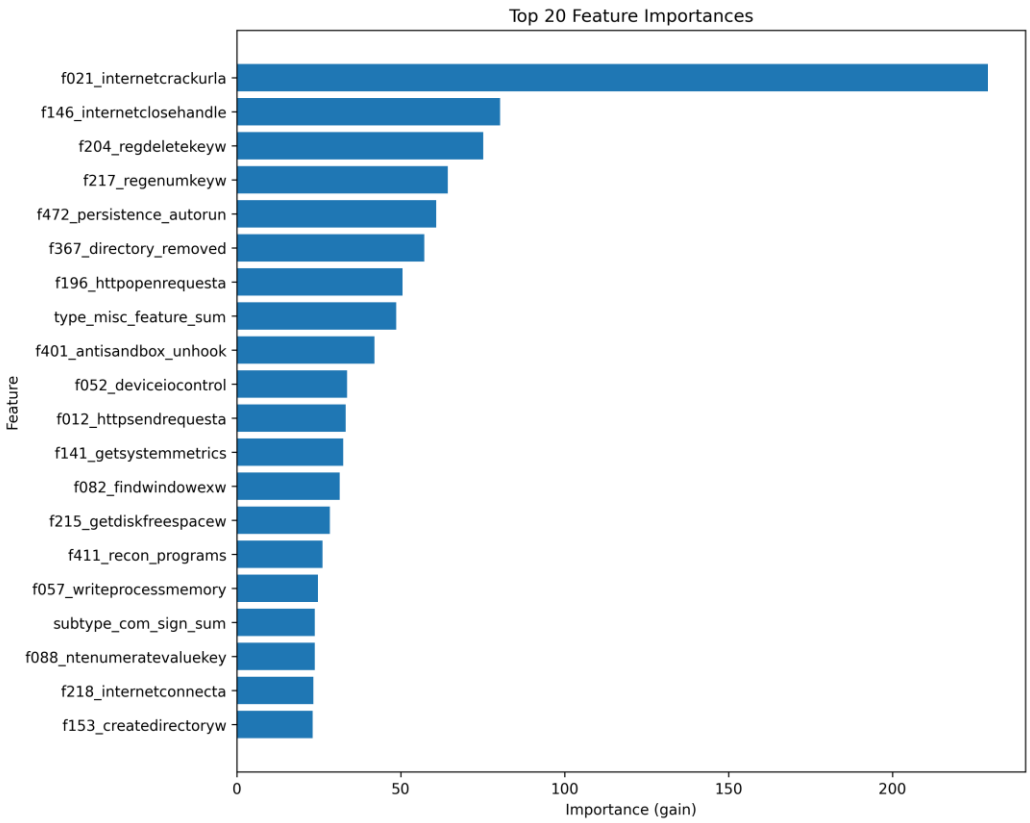
The final model is a gradient-boosted tree classifier using XGBClassifier. Gradient-boosted trees work well with structured tabular data and sparse behavioural features and so this model was ideal. The model uses the mapped raw features plus the engineered aggregate features. The XGBoost parameters controlled the number of boosted trees, tree depth, learning rate, and row and feature sampling used during training. risk_score was excluded because it was used to create the target, risk_level was excluded because it is the prediction target, and file_month was excluded because it was only used to create the time-based split. The data was split by month, using the first 60% for training, the next 20% for validation and the final 20% for testing. This gives a realistic deployment scenario where older application behaviour is used to predict later unseen applications. Although the classes are imbalanced, there was little drift across the splits. With train at 90.25% high-risk, validation at 90.76% high-risk and test at 90.55% high-risk.

Instead of hardcoding the classification threshold, the classification threshold was selected from the validation precision recall curve. The thresholds had to achieve at least 90% precision and 90% recall for the high-risk class. The final threshold was approximately 0.5142 and was chosen because it maximised high-risk F1 among the valid thresholds. Because of this use case, an antivirus, the high-risk class is the most important class, as missing a malicious application would be a false negative and is more costly than incorrectly flagging a safe application. To address the class imbalance, a scale_pos_weight XGBoost model was also tested. Since high risk was the majority class, scale_pos_weight was set below 1 using the training class ratio, with a value of 0.1080. The weighted model slightly improved the validations' low-risk recall from 41% to 44%. However, it reduced low-risk precision from 61% to 60% and had a slightly worse high-risk recall and high-risk F1. Thus, the baseline XGBoost model was selected because the main priority is high-risk performance. Low-risk recall remains weak, meaning the model may still flag a substantial number of benign applications as high risk, this is a trade-off from prioritising high-risk detection in this use case. The table below shows the performance of the final model on the validation split, it got 94% precision, 97% recall, and 96% F1 for the high-risk class. This meets the required targets of 90% precision and recall for the high-risk class. The test split was not used for threshold selection or model selection, it was only evaluated after the final model and threshold was chosen. On the test split, the final model achieved 92% precision and 94% recall for the high-risk class. The final model, selected threshold, mapper,

feature columns, and model name are packaged into packaged_model.pkl using Python's pickle package. Full train, validation, test, baseline, weighted-model, and threshold-search outputs are saved in the data/processed folder.

Split	Class	Precision	Recall	F1 Score	Support
Validation	Low risk, class 0	61%	41%	49%	1,936
Validation	High risk, class 1	94%	97%	96%	19,022

Feature Importance Plot



From the final model, the plot above shows the top 20 features with the highest importance gain. Many of the highest importance features are associated with malware related behaviours. For example, internetcrackurla, httpopenrequesta, httpsendrequesta and internetconnecta are network API features and they suggest external communication behaviour and can be associated with command-and-control activity or data exfiltration. Additionally, regdeletekeyw, regenumkeyw and persistence_autorun are registry and persistence features, which are associated with malware behaviours that modify system settings or attempt to survive restarts. Furthermore, antisandbox_unhook, deviceiocontrol and writeprocessmemory, are linked to anti-analysis, low-level system interaction or process manipulation. However, these features alone aren't proof of malicious behaviour, but it does suggest that the model is using relevant behavioural signals when classifying application risk.